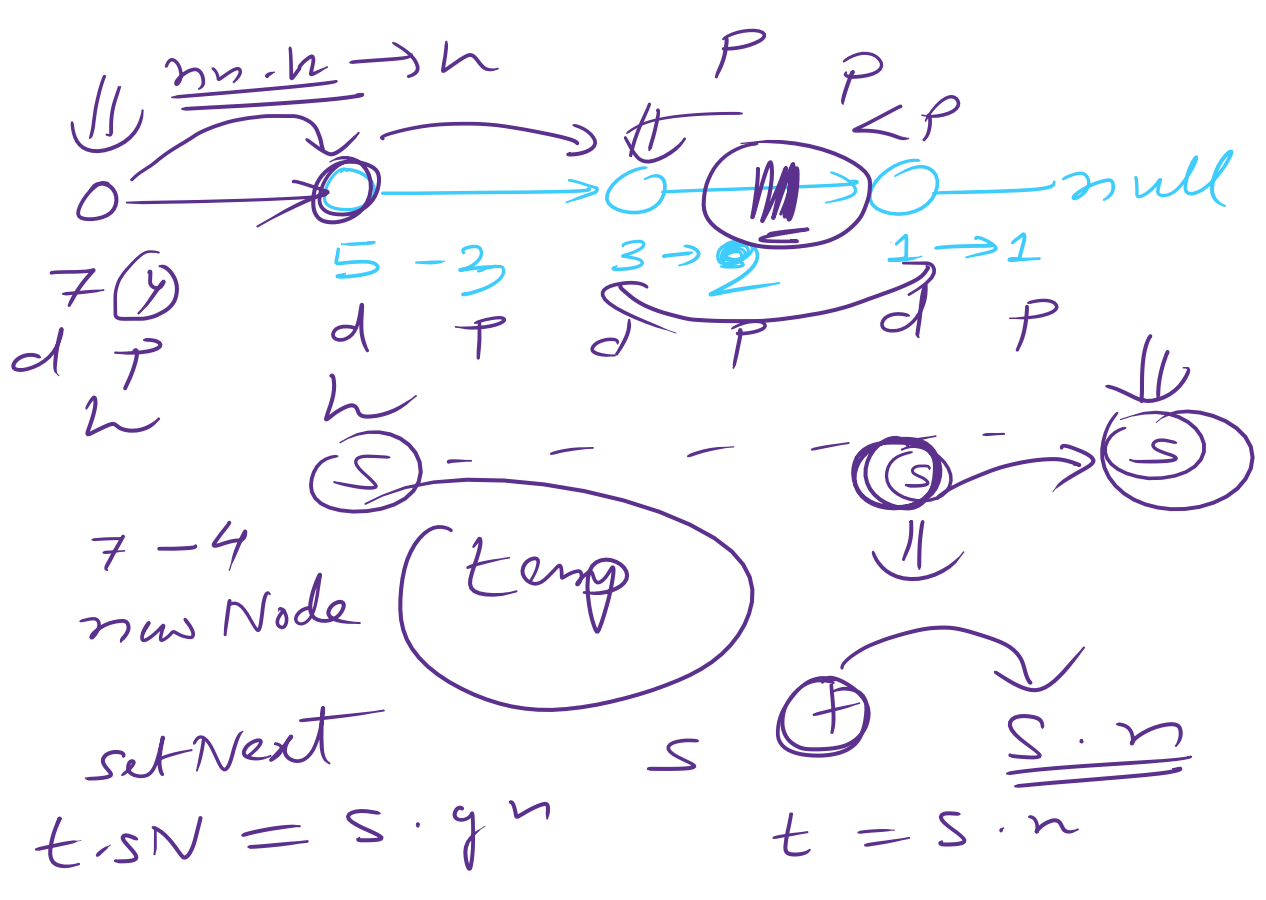
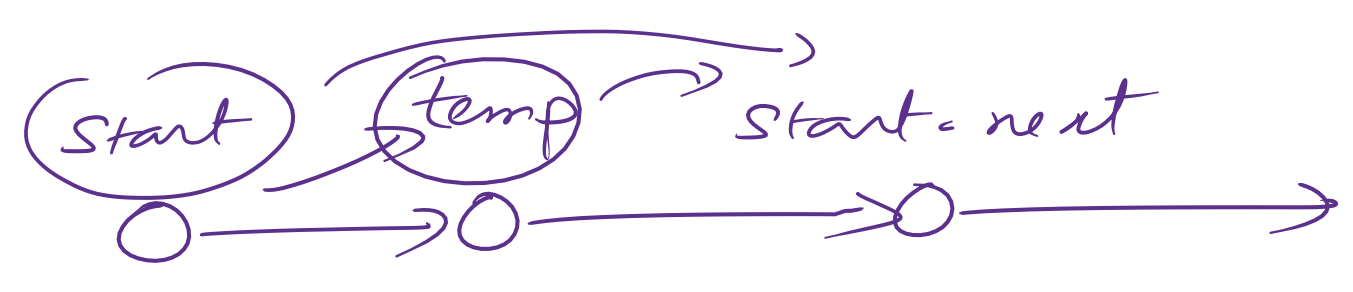

nn.n → n
P
P < P
null
7 (4)
5 − 3
3 → 2
1 → 1
d P
S
temp
7 − 4
new Node
setNext
t.sN = s.gn
s
t
s.n
t = s.n


Start
temp
start.next

temp.setNext(Start.getNext());

Start.setNext(temp);

Introduction to Graphs: →

Binary Search Application:

(i) First Occurrence
(ii) Last Occurrence
(iii) Total Occurrence (L − F + 1)

[1, 2, ③, ③, ③, 4, 5]
0 1 2 3 4 5 6

3 → FO → 2
3 → LO → 4   LO − FO + 1
3 → TO →   4 − 2 + 1
2 + 1 = ③

* Find the missing element inside a sorted array using Binary Search:

Note: 1 to n

arr[8] = [1, 2, 3, 5, 6, 7, 8] → ④

arr[8] = [1, 2, 3, 4, 5, 6, 7] → ⑧

[len + 1]
7 + 1 = 8   Binary Search

ind Ele = ind + 1 ←

a[i] = i+1

0+1	1+1	2+1	3+1	×	×	×
1	2	3	5	6	7	8
0	1	2	3	4	5	6

s   e

1 to n

mid = (0+6)/2 = ③

arr[3] = 3+1

① m == 0   m+1

5 ≠ 3+1 = 4 ←

-ve ind ←   return (m+1)   (3+1)

e = m − 1;

m == 0
return m+1
0+1 = ①

arr[m−1] = m = ④

arr[3−1] = 3

arr[2] = 3

if left part:

① mid == 0
return mid + 1

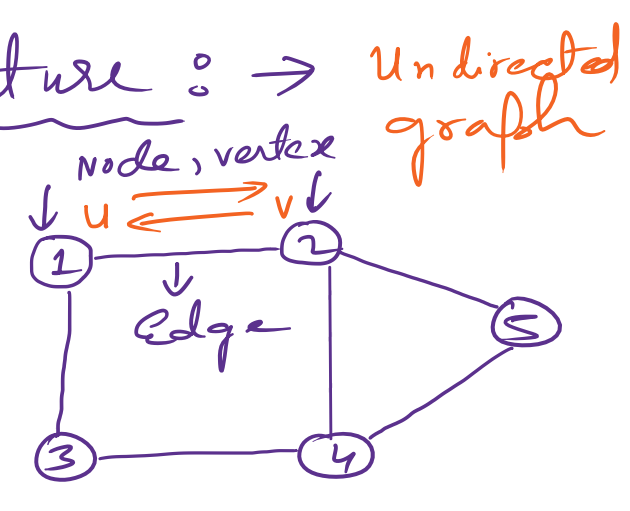
(ii) arr[mid − 1] == mid
return mid + 1

e = mid − 1;

arr[m] = m + 1

Right Side s = m + 1

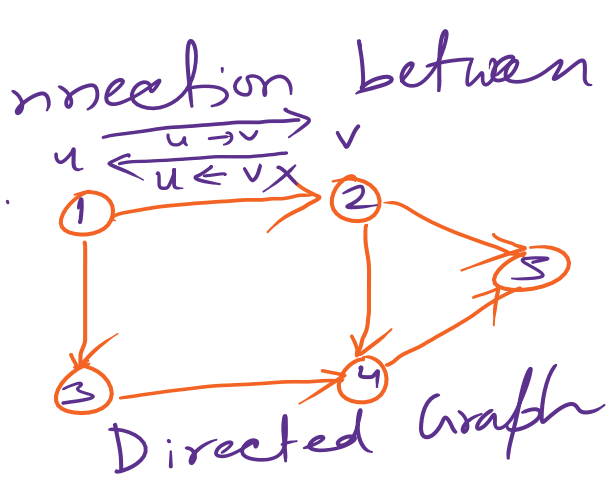
Graph Data Structure: → Undirected graph


Node, vertex
u
v
Edge
1
2
3
4
5

① Nodes, Vertex
(ii) Edges

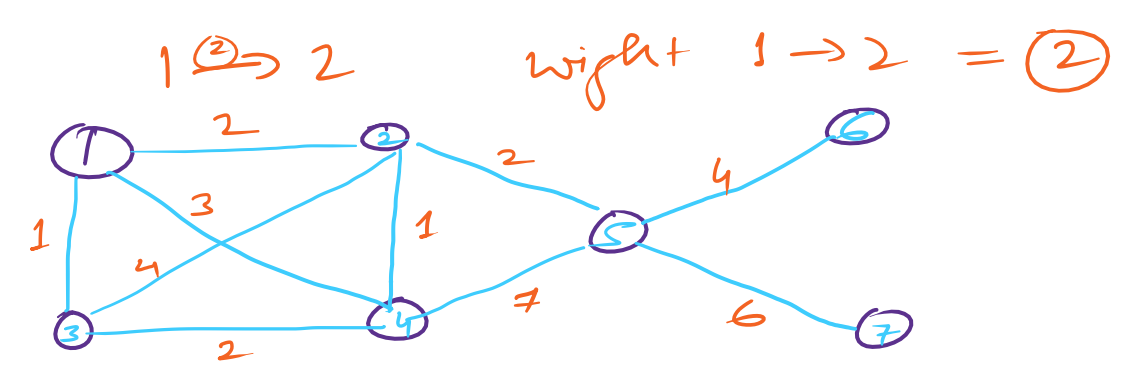
Node → Entity where we can store data or value.

Edge → It is the connection between two nodes/vertices.

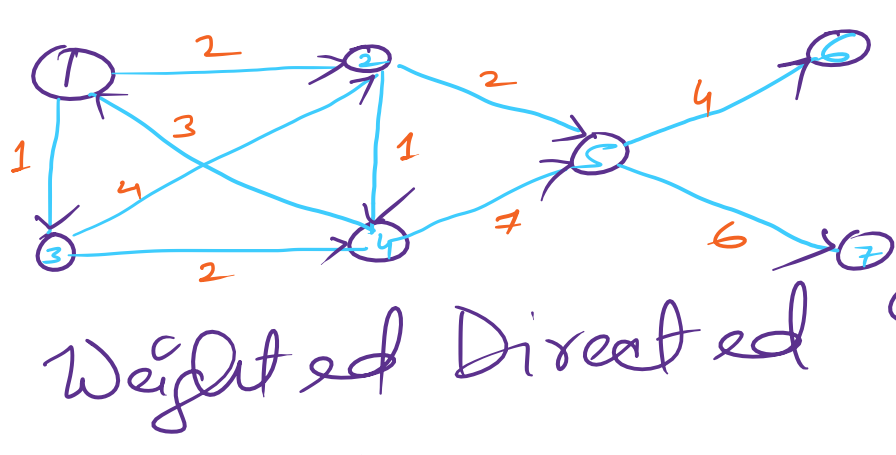

u
v
u → v
u ← v ×
1
2
3
4
5

Directed Graph

1 ② 2   weight 1 → 2 = ②

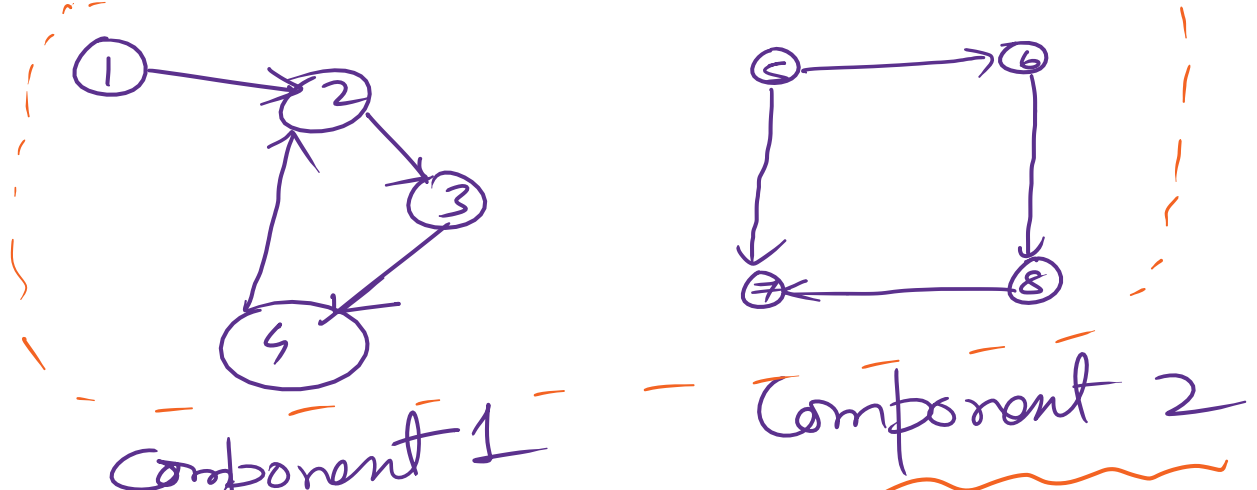

2
3
1
4
1
2
2
4
7
6
1
2
3
4
5
6
7

weights → Undirected Weighted Graph


2
3
1
4
1
2
2
4
7
6

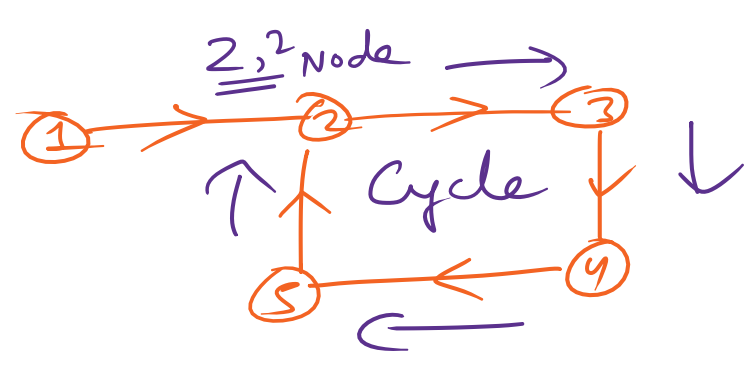
Weighted Directed Graph

Components

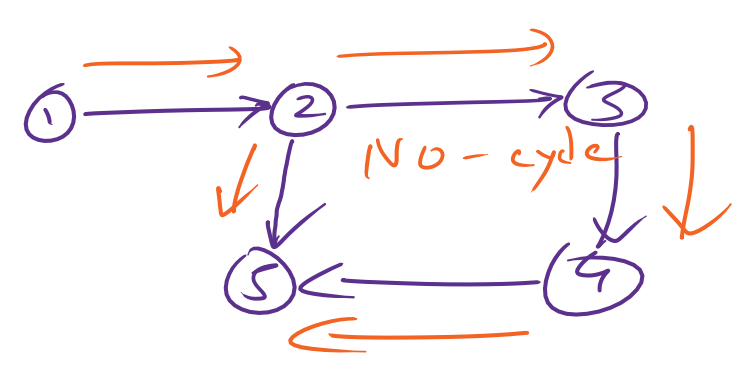


Component 1   Component 2

Disconnected Graph

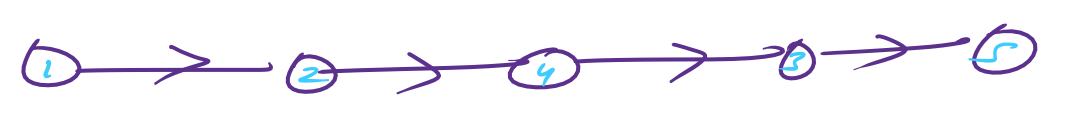

2,? Node
1
2
3
4
5
Cycle

Cyclic Graph


1
2
3
4
5
NO-cycle

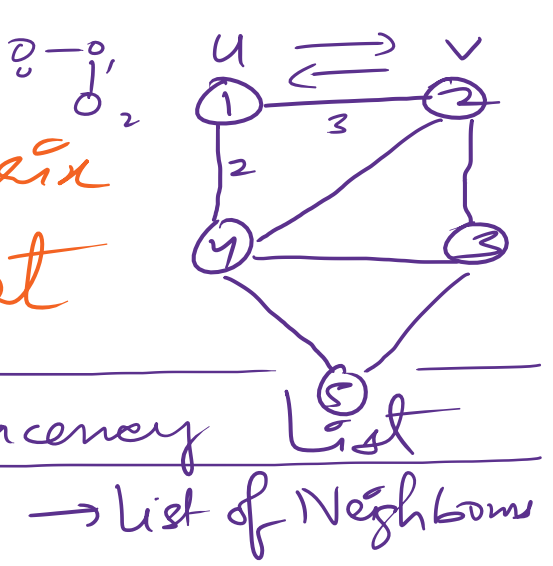
Acyclic Graph

Acyclic Graph



Graph Representation:

① Adjacency Matrix
(ii) Adjacency List


0-0, 0-1, 0-2
u
v
1
2
3
4
5
2
3

Adjacency Matrix

	0	1	2	3	4	5
0						
1		0	1	0	1	0
2		1	0	1	1	0
3		0	1	0	1	1
4		1	1	1	0	1
5		0	0	1	1	0

Adjacency List

Nodes	→ List of Neighbours
0	→
1	→ 2, 4
2	→ 1, 3, 4
3	→ 2, 4, 5
4	→ 1, 2, 3, 5
5	→ 3, 4

set<>, unordered map<>
AO

C++, OOPs, Exception Handling
Arrays
Searching & Sorting → Quick Sort → Heap Sort
Stacks & Queues   STL
Linked List → S, D, C
Trees → BT, BST, AVL, Heap
Priority Queues   CBT

Graphs → Representation → AM AL   ****
Traversal → DFS, BFS   [2010 2013]
Cycle Detection → DFS, BFS
Algos → Djikstra's, MST → Kruskal's

Recursion → Back Tracking → ④ Training
T M SO → Dynamic Programming (5−6)
① (ii) (iii) → Bit Masking
→ Problem Solving

(30) questions